

TravelMate Bildlade-Flow Analyse

Stand: aktuelle Repository-Analyse ohne Codeänderung.

Ziel: erklären, warum echte Aktivitätsbilder kurz sichtbar sein können und danach wieder durch Kategorie-Platzhalter ersetzt werden.

Kernaussage

Der Bildlade-Flow hat aktuell zwei Wahrheiten: lokale Bilder in ActivityGallery und zentrale Bilder im Trip-State aus useTripWorkspace. Wenn echte Bilder nur lokal ankommen, gehen sie beim Wechsel von Aktivität, Tag oder nach einem späteren Trip-Update wieder verloren. Dann greift die Galerie erneut auf Platzhalter zurück.

1. Wo entsteht der finale Bild-Slide-State

Der finale Slide-State entsteht im Frontend in frontend/components/ActivityGallery.vue. Die Galerie bekommt von DayOrbitCard.vue eine einzelne TripActivity. Intern baut ActivityGallery daraus GallerySlide-Objekte mit realImage und fallbackImage.

```
<ActivityGallery :key="(selectedActivity || day.activities[0]).id"
:activity="selectedActivity || day.activities[0]" :city="city" />
```

Es gibt zwei Fallback-Orte: normale Kategorie-Platzhalter in ActivityGallery.vue und einen zusätzlichen Notfall-Fallback in DayOrbitCard.vue, falls die Galerie selbst einen Fehler wirft.

2. Prioritätslogik in ActivityGallery

Die Galerie sammelt Bilder aus propImages und localImages. propImages kommen aus props.activity.activity.images, also aus dem zentralen Trip-State. localImages kommen aus enrichedImages, also aus einem lokalen Galerie-Request.

```
const propImages = computed(() =>
Array.isArray(props.activity.activity.images) ?
props.activity.activity.images : [])
const localImages = computed(() => Array.isArray(enrichedImages.value) ?
enrichedImages.value : [])
```

Wenn realImages vorhanden sind, erzeugt displaySlides echte Slides. Wenn realImages leer ist, erzeugt displaySlides genau einen Fallback-Slide. Ein Placeholder erscheint also wieder, sobald realImages leer wird oder ein echtes Bild als failed markiert wird.

3. Erfolgreich geladene Bilder

Das sichtbare echte Bild registriert seinen Ladeerfolg mit @load="markImageLoaded(slide.realImage.url)". Der Erfolg wird per URL in loadedImageUrls gespeichert. Diese URL ist meistens die interne Proxy-URL /activities/{activityId}/images/{imageId}/media.

Der Zustand loadedImageUrls lebt nur in der aktuellen Galerie-Instanz. Wird die Galerie neu gemountet, ist dieser Zustand weg.

4. Fehlgeschlagene Bilder

Fehler werden ebenfalls pro URL in failedImageUrls gespeichert. Innerhalb derselben Galerie-Instanz kann ein bereits erfolgreich geladenes Bild nicht mehr als failed gelten. Nach einem Remount ist dieser Schutz aber weg, weil loadedImageUrls neu startet.

5. Wo neue Bilder in den zentralen Trip-State geschrieben werden

Der zentrale Merge passiert in frontend/composables/useTripWorkspace.ts in `updateActivityImages(activityId, images)`. Dort werden Bilder fuer alle passenden Aktivitaeten im aktuellen `trip.value` gemerged.

```
if (item.activity.id === activityId) {  
  item.activity.images = mergeImages(item.activity.images, images)  
}
```

Wichtig: ActivityGallery.vue schreibt seine lokal geladenen Bilder aktuell nicht zuverlaessig in diesen zentralen Trip-State. Dort wird nur `enrichedImages.value = incomingImages` gesetzt. Das erklart, warum Bilder kurz sichtbar sein koennen, aber nach einem Remount verschwinden.

6. Hintergrund-Preloader

Der Preloader in `useTripWorkspace` laedt den aktiven Tag plus zwei Folgetage. Eine Aktivitaet gilt als erfolgreich, wenn mindestens ein Bild vorgeladen werden konnte. Der Preloader-Status wird aktuell aber nicht direkt von der Galerie fuer die Anzeigeentscheidung genutzt.

7. Doppelte Bildlade-Mechanismen

Es gibt zwei konkurrierende Wege: ActivityGallery.vue ruft beim Mount `loadImages()` auf, und `useTripWorkspace.ts` ruft `preloadUpcomingImages()` auf. Auf der Trip-Seite wird der Preloader beim Mount und beim Tagwechsel gestartet. Dadurch koennen Galerie und Preloader denselben Backend-Endpunkt fast gleichzeitig treffen.

8. Watcher und Remounts

Beim Aktivitaetswechsel loescht ActivityGallery den lokalen Zustand: `activeSlide`, `enrichedImages`, `failedImageUrls`, `loadedImageUrls` und `imageLoadStatus`. Zusaetzlich wird die Galerie ueber den Key der ausgewaehlten TripActivity neu erzeugt.

```
watch(() => props.activity.id, () => {  
  enrichedImages.value = null  
  failedImageUrls.value = []  
  loadedImageUrls.value = []  
  loadImages()  
})
```

9. CSS- und Rendering-Logik

Aktuell liegt das echte Bild ueber dem Platzhalter: `.gallery-fallback-image` hat `z-index: 1` und `.gallery-real-image` hat `z-index: 2`. CSS sieht im aktuellen Stand nicht nach der Hauptursache aus. Der Platzhalter wird sichtbar, wenn das echte Bild nicht mehr im DOM ist oder als failed gilt.

10. Backend-Endpunkt fuer Bild-Enrichment

Der Endpunkt `POST /activities/{id}/images` liegt in `ActivityResource.java` und ruft `imageService.ensureImages(id)` auf. `GooglePlacesImageService` ist grundsaeztlich idempotent: Wenn `activity.images` nicht leer ist, werden vorhandene Bilder zurueckgegeben.

Restrisiko: Bei parallelen Requests koennen zwei Requests gleichzeitig sehen, dass noch keine Bilder existieren. Wenn einer beim Speichern wegen Parallelitaet scheitert, landet er im Catch und gibt List.of() zurueck.

11. Interne Media-URL

Das Frontend bekommt meist keine direkte Google-URL, sondern `/activities/{activityId}/images/{imageId}/media`. Der Backend-Proxy loest diese URL bei jedem Bildaufruf ueber Google auf und gibt einen temporaeren Redirect zurueck. Moegliche Fehlerquellen sind Google-Timeouts, 403 durch Key-Restrictions, ungueltige Photo-Referenzen oder langsame Redirects.

12. Wahrscheinlichkeitsbewertung

Sehr hoch: lokaler Galerie-State geht verloren. Hoch: State-Overwrite mit alter Activity ohne Bilder. Mittel bis hoch: doppelte Requests und Race Conditions. Mittel: `onError` nach `onLoad` nach Remount. Niedrig im aktuellen Stand: CSS versteckt echte Bilder. Mittel: leere API-Antworten und lokale Placeholder-Entscheidungen.

13. Wahrscheinlichste Root Cause

Echte Bilder werden teilweise nur lokal in `ActivityGallery.enrichedImages` gehalten. Dieser lokale Zustand wird beim Aktivitaetswechsel, Tageswechsel oder Remount geloescht. Wenn der zentrale Trip-State keine Bilder hat oder spaeter ueberschrieben wird, zeigt die Galerie wieder Platzhalter.

Betroffene Dateien

`frontend/components/ActivityGallery.vue`; `frontend/composables/useTripWorkspace.ts`;
`frontend/components/DayOrbitCard.vue`; `frontend/pages/trips/[id].vue`;
`backend/src/main/java/de/travelmate/activity/ActivityResource.java`;
`backend/src/main/java/de/travelmate/activity/GooglePlacesImageService.java`;
`backend/src/main/java/de/travelmate/activity/ActivityImageDto.java`.

Minimaler Fix-Vorschlag

1. `useTripWorkspace` wird der zentrale Besitzer echter Bilddaten. 2. Wenn `ActivityGallery` Bilder selbst laedt, muss sie diese in den zentralen Trip-State mergen. 3. `replaceTrip(updated)` darf vorhandene echte Bilder nicht durch `images: []` ersetzen. 4. Backend soll nach einem parallelen Enrichment-Fehler vorhandene Bilder erneut aus der DB lesen. 5. Wenn echte Bilder vorhanden sind, darf der Platzhalter nie den finalen Slide-State ersetzen.

Risiken des Fixes

Wenn Bilder zu aggressiv erhalten werden, koennen alte Bilder laenger im Client bleiben. Ein explizites Entfernen von Bildern braechte spaeter ein eigenes Signal. Wenn Galerie und Preloader weiter parallel laden, bleiben unnoetige Requests moeglich. Backend-Reqquery nach Parallelfehlern ist sicherer, aber etwas mehr DB-Arbeit.

Empfohlene Tests und Logs

Frontend: Log bei Galerie-Mount mit Activity-ID, `proplImages.length`, `localImages.length` und `reallImages.length`; Log bei `markImageLoaded` und `markImageFailed`; Log bei `replaceTrip` mit Bilderzahl vor/nach dem Ersetzen. Backend: Log bei `ensureImages` mit vorhandenen Bildern, Google-Match und gespeicherter Bildanzahl. Test: Bild erscheint, Aktivitaet wechseln, zurueckwechseln, echtes Bild muss bleiben. Tag wechseln und zurueck, echtes Bild muss bleiben.